

Computer Vision Practical Report:

CROPPING,COPYING,PASTING IMAGE INSIDE ANOTHER IMAGE

1. AIM

To crop a Region of Interest (ROI) from a source image, copy it, and paste it onto a target destination image using NumPy array slicing in OpenCV, display the results, and save the output image.

2. PROCEDURE

- Load the source image and target image using `cv2.imread()`.
- Define the coordinates (y_1, y_2, x_1, x_2) for the Region of Interest (ROI) and crop it using NumPy array slicing (`roi = source[y1:y2, x1:x2]`).
- Define the insertion offsets $(y_{\text{start}}, x_{\text{start}})$ on the target image matching the dimensions of the cropped ROI.
- Paste the cropped ROI onto the target image by assigning the ROI array slice directly to the target array region (`target[y:y+h, x:x+w] = roi`).
- Display the original, cropped ROI, and final pasted images side-by-side using Matplotlib, and save the result using `cv2.imwrite()`.

3. PROGRAM CODE

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image
img_bgr = cv2.imread('E28-IMG.jpg')
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 2. Define Boundary Extraction Kernel (3x3 Laplacian filter)
```

```
kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
], dtype=np.float32)

# 3. Apply 2D convolution to find boundary
boundary = cv2.filter2D(gray, -1, kernel)

# 4. Display on a single page
fig, axes = plt.subplots(1, 2, figsize=(8, 4))

axes[0].imshow(img_rgb)
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(boundary, cmap='gray')
axes[1].set_title('Extracted Boundary')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

4. OUTPUT

Original Image



Extracted Boundary



5. RESULT

The Python script executed successfully. The input image was loaded.